Logic Flaws

Complete Security Guide — Business Logic Vulnerabilities • Discovery • Exploitation & Prevention

1. What Are Logic Flaws

Application logic errors occur when developers design a system that technically works as intended but can be abused due to flawed assumptions about user behavior, input, or workflow. These are not always caused by misconfigurations or missing sanitization — they're errors in **how the app thinks**.

2. Why They're Hard to Find

Logic flaws:

- Are not detected by scanners
- Often require deep knowledge of the application's functionality
- Vary significantly between different applications
- Demand creativity, patience, and domain understanding

Unlike XSS or SQLi, there is no universal payload. You need to study the business logic and look for **abuse scenarios**.

3. Examples of Logic Flaws

- **Race Conditions:** Performing the same action in parallel results in unauthorized access or money transfer
- **Broken Price Validation:** Manipulating product prices in client-side data and bypassing backend checks
- **Account Takeovers via Password Reset Flows:** Predictable tokens, missing verification steps, or misuse of user identifiers
- **Coupon Abuse:** Reusing or stacking promo codes
- **Authentication Bypass via Logic Abuse:** Skipping steps, guessing tokens, or reusing sessions
- **Improper Workflow Checks:** User is allowed to perform admin actions via broken role validation

4. Real-World Scenarios

- **Booking Manipulation:** Booking multiple rooms or flights for free by abusing how availability is calculated
- **Points/Funds Manipulation:** Redeeming the same loyalty points multiple times due to missing transaction locking
- **Business Process Bypass:** Finalizing purchases without completing full payment flow

5. How to Discover Logic Issues

- **Think Like a Malicious User:** "What would I do if I had no morals and access to this app?"
- **Map the Workflow:** Visualize the steps required to complete actions, then test each step in isolation
- **Test State Transitions:** Can steps be repeated, skipped, or performed in a different order?
- **Fuzz Identifiers:** Test how the system handles user IDs, order numbers, tokens, etc.

Ask:

- Can I trick the system into doing something it shouldn't?
- Is it making assumptions that I can violate?
- Is the server verifying or blindly trusting client-side state?

6. Tools and Techniques

Logic bugs are mostly manual:

- **Burp Suite (Repeater, Intruder):** Replay requests with altered flow
- **Race condition tools:** race-the-web, Turbo Intruder
- **Logging and Recording:** Use plugins to capture complete user actions and replicate the workflow

Also:

- Observe response times and inconsistencies
- Compare flows of normal vs. abnormal usage
- Replay sequences in unexpected orders

7. Bug Bounty Report Tips

When reporting logic issues:

- Carefully explain the **business context** and **impact**
- Include **step-by-step reproduction**
- Demonstrate how it can be **abused at scale**
- Emphasize **financial, privacy, or authorization impact**

Example:

> *"By replaying step 3 of the checkout without completing step 2, I was able to purchase the product without being charged."*

8. Logic vs Exploit

Logic bugs often:

- Have no visible errors
- Do not crash the app

- Don't trigger security alerts

That's why they're **prized** in bug bounty — many go unnoticed by security teams until reported.

9. Not Always Technical

Some of the best logic bugs are found by understanding:

- Business workflows
- Compliance and policy assumptions
- How real users interact with the platform

You don't need advanced tooling — you need critical thinking and creativity.

10. Common Categories to Focus On

- Authentication flows
- Payment and subscription logic
- User roles and permissions
- Resource creation/deletion (race conditions)
- Rate limits and abuse controls
- Redemption/reward systems
- Workflow transitions and edge cases

Conclusion

Logic flaws require you to think outside the box. They reward observation, creativity, and an attacker's mindset. Learn how apps behave — not just how they're built — and you'll find vulnerabilities no scanner ever will.